

CHL5208 Code

Peiqing Yu

2026/7/13

1. Simulation Scenario definition

```
get_scenario_params <- function(scenario) {  
  
  params <- list(  
    beta_trt = 0,  
    utility_fun = function(t) 0,  
    base_fun = function(t) 0.65 + 0.15 * (1 - exp(-t / 8))  
  )  
  
  if (scenario == "A") {  
    # Null: no survival or utility difference  
    params$beta_trt <- 0  
    params$utility_fun <- function(t) 0  
  }  
  
  if (scenario == "B") {  
    # Survival benefit only  
    # HR = exp(-0.80) = 0.45  
    params$beta_trt <- -0.80  
    params$utility_fun <- function(t) 0  
  }  
  
  if (scenario == "C") {  
    # Utility benefit only  
    params$beta_trt <- 0  
    params$utility_fun <- function(t) 0.18  
  }  
  
  if (scenario == "D") {  
    # Early harm, late benefit  
    params$beta_trt <- -0.40  
    params$utility_fun <- function(t) {  
      ifelse(t <= 12, -0.08,  
            ifelse(t <= 30, 0.04, 0.24))  
    }  
  }  
  
  if (scenario == "E") {  
    # Longer survival, worse QoL
```

```

    params$beta_trt <- -0.75
    params$utility_fun <- function(t) -0.12
  }

  if (scenario == "F") {
    # Better QoL, shorter survival
    params$beta_trt <- 0.60
    params$utility_fun <- function(t) 0.18
  }

  if (scenario == "G") {
    # Benefit in both survival and utility
    # This is your strongest demonstration of QALY/HUS
    params$beta_trt <- -0.65
    params$utility_fun <- function(t) 0.16
  }

  return(params)
}

```

2. Data generation

- 2.1 Covariates
- 2.2 Treatment assignment
- 2.3 Survival outcome
- 2.4 Utility trajectories
- 2.5 Missingness

```

simulate_data <- function(
  n0 = 150,
  n1 = 150,
  tt = 60,
  scenario = "A",
  observational = TRUE,

  # treatment assignment model
  # severity is the main source of observational confounding
  alpha = c(alpha0 = 0, age = 0.00, sex = 0.00, sev = 0.65),

  # survival model
  beta = c(beta0 = -4.0, age = 0.025, sex = 0.025, sev = 0.6),
  shape = 1.2,

  # utility model
  theta = c(age = -0.01, sex = -0.01, sev = -0.1),
  sigma_subject = 0.08,
  sigma_noise = 0.04,

  # missingness
  p_miss = 0.05,

  seed = NULL

```

```

) {

  if (!is.null(seed)) set.seed(seed)

  params <- get_scenario_params(scenario)

  #####
  # 1. Generate large covariate pool
  #####

  n_pool <- 10000

  age <- runif(n_pool, 35, 80)
  sex <- rbinom(n_pool, 1, 0.5)
  sev <- rnorm(n_pool, 0, 1)

  # Center and scale covariates
  age_c <- (age - 57.5) / 10
  sex_c <- sex - 0.5

  #####
  # 2. Treatment assignment
  #####

  if (observational) {
    linpred_trt <- alpha["alpha0"] +
      alpha["age"] * age_c +
      alpha["sex"] * sex_c +
      alpha["sev"] * sev

    probb_trt <- plogis(linpred_trt)
    trt_all <- rbinom(n_pool, 1, probb_trt)
  } else {
    trt_all <- rbinom(n_pool, 1, 0.5)
  }

  id1_pool <- which(trt_all == 1)
  id0_pool <- which(trt_all == 0)

  if (length(id1_pool) < n1 | length(id0_pool) < n0) {
    stop("Not enough subjects in one treatment group. Adjust alpha or n_pool.")
  }

  id1 <- sample(id1_pool, n1)
  id0 <- sample(id0_pool, n0)

  age1 <- age[id1]
  age0 <- age[id0]

  sex1 <- sex[id1]
  sex0 <- sex[id0]

  sev1 <- sev[id1]

```

```

sev0 <- sev[id0]

#####
# 3. Survival model
#####

lin0 <- beta["beta0"] +
  beta["age"] * (age0 - 57.5)/10 +
  beta["sex"] * (sex0 - 0.5) +
  beta["sev"] * sev0

lin1 <- beta["beta0"] +
  beta["age"] * (age1 - 57.5)/10 +
  beta["sex"] * (sex1 - 0.5) +
  beta["sev"] * sev1 +
  params$beta_trt

lambda0 <- exp(lin0)
lambda1 <- exp(lin1)

T0 <- (-log(runif(n0)) / lambda0)^(1 / shape)
T1 <- (-log(runif(n1)) / lambda1)^(1 / shape)

# Mild censoring
C0 <- runif(n0, 0.8*tt, tt)
C1 <- runif(n1, 0.8*tt, tt)

time0 <- pmin(T0, C0, tt)
time1 <- pmin(T1, C1, tt)

event0 <- as.numeric(T0 <= C0 & T0 <= tt)
event1 <- as.numeric(T1 <= C1 & T1 <= tt)

time0 <- pmax(1, ceiling(time0))
time1 <- pmax(1, ceiling(time1))

S0 <- data.frame(time = time0, event = event0)
S1 <- data.frame(time = time1, event = event1)

#####
# 4. Utility trajectories
#####

make_utility <- function(n, age, sex, sev, S, trt) {

  U <- matrix(NA, nrow = n, ncol = tt)

  for (i in 1:n) {

    b_i <- rnorm(1, 0, sigma_subject)

    for (t in 1:tt) {

```

```

base_t <- params$base_fun(t)
trt_eff_t <- ifelse(trt == 1, params$utility_fun(t), 0)

u_it <- base_t +
  trt_eff_t +
  theta["age"] * (age[i] - 57.5)/10 +
  theta["sex"] * (sex[i] - 0.5) +
  theta["sev"] * sev[i] +
  b_i +
  rnorm(1, 0, sigma_noise)

u_it <- min(max(u_it, 0), 1)

# after death utility is zero
if (S$event[i] == 1 & t > S$time[i]) {
  u_it <- 0
}

U[i, t] <- u_it
}
}

return(U)
}

U0_full <- make_utility(n0, age0, sex0, sev0, S0, trt = 0)
U1_full <- make_utility(n1, age1, sex1, sev1, S1, trt = 1)

#####
# 5. Missingness
#####

U0 <- U0_full
U1 <- U1_full

for (i in 1:n0) {
  for (t in 1:tt) {
    if (!(S0$event[i] == 1 & t > S0$time[i])) {
      if (rbinom(1, 1, p_miss) == 1) U0[i, t] <- NA
    }
  }
}

for (i in 1:n1) {
  for (t in 1:tt) {
    if (!(S1$event[i] == 1 & t > S1$time[i])) {
      if (rbinom(1, 1, p_miss) == 1) U1[i, t] <- NA
    }
  }
}

#####
# 6. Return

```

```
#####

return(list(
  S_trt = S1,
  S_ctl = S0,
  U_trt = U1,
  U_ctl = U0,
  U_trt_full = U1_full,
  U_ctl_full = U0_full,
  age_trt = age1,
  age_ctl = age0,
  sex_trt = sex1,
  sex_ctl = sex0,
  sev_trt = sev1,
  sev_ctl = sev0,
  scenario = scenario
))
}
```

3. Missing data imputation

```
fillU <- function(Ut) {

  Uf <- Ut
  n <- length(Uf)

  which_na <- which(is.na(Uf))
  which_obs <- which(!is.na(Uf))

  # No missing
  if (length(which_na) == 0) return(Uf)

  # If all values are missing, use neutral utility
  if (length(which_obs) == 0) {
    Uf[] <- 0.5
    return(Uf)
  }

  for (hh in which_na) {

    if (hh < min(which_obs)) {
      Uf[hh] <- Uf[min(which_obs)]

    } else if (hh > max(which_obs)) {
      Uf[hh] <- Uf[max(which_obs)]

    } else {
      # Missing between observed values: linear interpolation
      prev_obs <- max(which_obs[which_obs < hh])
      next_obs <- min(which_obs[which_obs > hh])

      Uf[hh] <- Uf[prev_obs] +
```

```

      (Uf[next_obs] - Uf[prev_obs]) *
      (hh - prev_obs) / (next_obs - prev_obs)
    }
  }

  return(Uf)
}

impute_matrix <- function(U) {
  t(apply(U, 1, fillU))
}

```

4. IPTW adjustment

```

get_weights <- function(age, sex, sev, trt, trim = 10) {

  df <- data.frame(
    trt = trt,
    age_c = (age-57.5)/10,
    sex_c = sex-0.5,
    sev = sev
  )

  fit <- glm(trt ~ age_c + sex_c + sev, data = df, family = binomial)

  ps <- predict(fit, type = "response")

  p_trt <- mean(trt)

  w <- ifelse(
    trt == 1,
    p_trt / ps,
    (1 - p_trt) / (1 - ps)
  )

  w <- pmin(w, trim)

  return(list(
    weights = w,
    ps = ps,
    model = fit
  ))
}

```

5. QALY/HUS calculation

```

QALY_IPTW <- function(S, U, w, tt, time_weight = NULL,
                      lam_surv = 1, lam_util = 1) {

```

```

if (is.null(time_weight)) {
  time_weight <- rep(1, tt)
}

fit <- survfit(Surv(time, event) ~ 1, data = S, weights = w)

S_KM <- numeric(tt)

for (t in 1:tt) {
  if (length(fit$time) == 0) {
    S_KM[t] <- 1
  } else {
    idx <- which(fit$time <= t)
    S_KM[t] <- ifelse(length(idx) == 0, 1, fit$surv[max(idx)])
  }
}

U_bar <- numeric(tt)

for (t in 1:tt) {
  alive <- which(S$time >= t)

  if (length(alive) == 0) {
    U_bar[t] <- 0
  } else {
    obs <- alive[!is.na(U[alive, t])]

    if (length(obs) == 0) {
      U_bar[t] <- 0
    } else {
      U_bar[t] <- weighted.mean(U[obs, t], w[obs])
    }
  }
}

qaly <- sum(
  time_weight *
  (S_KM ^ lam_surv) *
  (U_bar ^ lam_util)
)

return(list(
  qaly = qaly,
  survival_curve = S_KM,
  utility_curve = U_bar
))
}

```

6. Statistical tests

- 6.1 Survival-only test

- 6.2 Utility-only test
- 6.3 QALY/HUS permutation test

```

# 6.1 survival only test
survival_test <- function(S1, S0, w1, w0) {

  S <- rbind(S1, S0)
  S$trt <- c(rep(1, nrow(S1)), rep(0, nrow(S0)))

  w <- c(w1, w0)

  fit <- coxph(
    Surv(time, event) ~ trt,
    data = S,
    weights = w,
    robust = TRUE
  )

  pval <- summary(fit)$coef["trt", "Pr(>|z|)"]

  return(pval)
}

# 6.2 utility only test
# find and compare subject level mean utility while alive
utility_test <- function(U1, U0, S1, S0, w1, w0, nperm = 500) {

  U1_imp <- impute_matrix(U1)
  U0_imp <- impute_matrix(U0)

  subject_alive_mean <- function(U, S) {

    out <- numeric(nrow(U))

    for (i in 1:nrow(U)) {
      alive_t <- which(1:ncol(U) <= S$time[i])
      out[i] <- mean(U[i, alive_t], na.rm = TRUE)
    }

    return(out)
  }

  m1 <- subject_alive_mean(U1_imp, S1)
  m0 <- subject_alive_mean(U0_imp, S0)

  diff_obs <- weighted.mean(m1, w1) - weighted.mean(m0, w0)

  m_all <- c(m1, m0)
  w_all <- c(w1, w0)

  n1 <- length(m1)
  n <- length(m_all)

  diffs <- numeric(nperm)

```

```

for (b in 1:nperm) {

  idx <- sample(1:n, n1)

  diffs[b] <-
    weighted.mean(m_all[idx], w_all[idx]) -
    weighted.mean(m_all[-idx], w_all[-idx])
}

pval <- (sum(abs(diffs) >= abs(diff_obs)) + 1) / (nperm + 1)

return(pval)
}
# 6.3 QALY/HUS permutation test
qaly_test <- function(dat, nperm = 500, tt = 60,
                      lam_surv = 1, lam_util = 1,
                      time_weight = NULL) {

  S1 <- dat$S_trt
  S0 <- dat$S_ctl

  U1 <- impute_matrix(dat$U_trt)
  U0 <- impute_matrix(dat$U_ctl)

  age <- c(dat$age_trt, dat$age_ctl)
  sex <- c(dat$sex_trt, dat$sex_ctl)
  sev <- c(dat$sev_trt, dat$sev_ctl)

  trt <- c(rep(1, nrow(S1)), rep(0, nrow(S0)))

  # Estimate IPTW once from observed treatment assignment
  # This keeps permutation less conservative than recomputing weights each time
  w <- get_weights(age, sex, sev, trt)$weights

  w1 <- w[trt == 1]
  w0 <- w[trt == 0]

  q1 <- QALY_IPTW(S1, U1, w1, tt, time_weight, lam_surv, lam_util)$qaly
  q0 <- QALY_IPTW(S0, U0, w0, tt, time_weight, lam_surv, lam_util)$qaly

  diff_obs <- q1 - q0

  S_all <- rbind(S1, S0)
  U_all <- rbind(U1, U0)

  n1 <- nrow(S1)
  n <- nrow(S_all)

  diff_perm <- numeric(nperm)

  for (b in 1:nperm) {

    idx <- sample(1:n, n1)

```

```

q1p <- QALY_IPTW(
  S_all[idx, ],
  U_all[idx, ],
  w[idx],
  tt,
  time_weight,
  lam_surv,
  lam_util
)$qaly

q0p <- QALY_IPTW(
  S_all[-idx, ],
  U_all[-idx, ],
  w[-idx],
  tt,
  time_weight,
  lam_surv,
  lam_util
)$qaly

diff_perm[b] <- q1p - q0p
}

pval <- (sum(abs(diff_perm) >= abs(diff_obs)) + 1) / (nperm + 1)

return(list(
  qaly_diff = diff_obs,
  qaly_p = pval,
  qaly_trt = q1,
  qaly_ctl = q0,
  perm_distribution = diff_perm
))
}

```

7. One simulation replicate

```

run_one_simulation <- function(
  scenario,
  n0 = 150,
  n1 = 150,
  tt = 60,
  nperm = 500,
  seed = NULL
) {

  dat <- simulate_data(
    n0 = n0,
    n1 = n1,
    tt = tt,
    scenario = scenario,
    seed = seed
  )
}

```

```

)

age <- c(dat$age_trt, dat$age_ctl)
sex <- c(dat$sex_trt, dat$sex_ctl)
sev <- c(dat$sev_trt, dat$sev_ctl)
trt <- c(rep(1, n1), rep(0, n0))

w <- get_weights(age, sex, sev, trt)$weights

w1 <- w[trt == 1]
w0 <- w[trt == 0]

surv_p <- survival_test(dat$S_trt, dat$S_ctl, w1, w0)

util_p <- utility_test(
  dat$U_trt,
  dat$U_ctl,
  dat$S_trt,
  dat$S_ctl,
  w1,
  w0,
  nperm = nperm
)

qaly_obj <- qaly_test(
  dat = dat,
  nperm = nperm,
  tt = tt
)

balance <- data.frame(
  covariate = c("age", "sex", "severity"),
  trt_mean = c(mean(dat$age_trt), mean(dat$sex_trt), mean(dat$sev_trt)),
  ctl_mean = c(mean(dat$age_ctl), mean(dat$sex_ctl), mean(dat$sev_ctl)),
  trt_wmean = c(
    weighted.mean(dat$age_trt, w1),
    weighted.mean(dat$sex_trt, w1),
    weighted.mean(dat$sev_trt, w1)
  ),
  ctl_wmean = c(
    weighted.mean(dat$age_ctl, w0),
    weighted.mean(dat$sex_ctl, w0),
    weighted.mean(dat$sev_ctl, w0)
  )
)

return(list(
  scenario = scenario,
  surv_p = surv_p,
  util_p = util_p,
  qaly_p = qaly_obj$qaly_p,
  qaly_diff = qaly_obj$qaly_diff,

```

```

    balance = balance,
    data = dat
  ))
}

run_all_scenarios_once <- function(
  scenarios = c("A", "B", "C", "D", "E", "F", "G"),
  n0 = 100,
  n1 = 100,
  tt = 60,
  nperm = 500,
  seed = 123
) {

  results <- data.frame(
    scenario = scenarios,
    surv_p = NA_real_,
    util_p = NA_real_,
    qaly_p = NA_real_,
    qaly_diff_months = NA_real_,
    qaly_diff_years = NA_real_,
    mean_sev_trt_raw = NA_real_,
    mean_sev_ctl_raw = NA_real_,
    mean_sev_trt_wt = NA_real_,
    mean_sev_ctl_wt = NA_real_,
    stringsAsFactors = FALSE
  )

  for (k in seq_along(scenarios)) {

    s <- scenarios[k]

    cat("Running scenario:", s, "\n")

    one <- run_one_simulation(
      scenario = s,
      n0 = n0,
      n1 = n1,
      tt = tt,
      nperm = nperm,
      seed = seed
    )

    results$surv_p[k] <- one$surv_p
    results$util_p[k] <- one$util_p
    results$qaly_p[k] <- one$qaly_p
    results$qaly_diff_months[k] <- one$qaly_diff
    results$qaly_diff_years[k] <- one$qaly_diff / 12

    balance <- one$balance

    sev_row <- balance[balance$covariate == "severity", ]
  }
}

```

```

    results$mean_sev_trt_raw[k] <- sev_row$trt_mean
    results$mean_sev_ctl_raw[k] <- sev_row$ctl_mean
    results$mean_sev_trt_wt[k] <- sev_row$trt_wmean
    results$mean_sev_ctl_wt[k] <- sev_row$ctl_wmean
  }

  results$urv_sig <- results$urv_p < 0.05
  results$util_sig <- results$util_p < 0.05
  results$qaly_sig <- results$qaly_p < 0.05

  results <- merge(
    results,
    expected_reference,
    by = "scenario",
    all.x = TRUE
  )

  return(results)
}

```

The expected p-value tables as follows

Running scenarios for one round of simulation

```

one_run_results <- run_all_scenarios_once(
  scenarios = c("A", "B", "C", "D", "E", "F", "G"),
  n0 = 150,
  n1 = 150,
  tt = 60,
  nperm = 500
)

```

```

## Running scenario: A
## Running scenario: B
## Running scenario: C
## Running scenario: D
## Running scenario: E
## Running scenario: F
## Running scenario: G

```

```
one_run_results
```

```

##   scenario      surv_p      util_p      qaly_p qaly_diff_months
## 1      A 1.244513e-01 0.451097804 0.133732535      2.991052
## 2      B 8.120224e-11 0.063872255 0.001996008      12.901067
## 3      C 1.244513e-01 0.001996008 0.003992016      6.573417
## 4      D 4.799815e-05 0.211576846 0.001996008      9.044609
## 5      E 7.189425e-10 0.001996008 0.001996008      7.887801
## 6      F 3.679671e-02 0.001996008 0.518962076     -1.169205
## 7      G 2.480560e-08 0.001996008 0.001996008     15.710442
##   qaly_diff_years mean_sev_trt_raw mean_sev_ctl_raw mean_sev_trt_wt
## 1      0.24925431      0.1933658      -0.2305269     -0.02686954
## 2      1.07508896      0.1933658      -0.2305269     -0.02686954

```

## 3	0.54778474	0.1933658	-0.2305269	-0.02686954
## 4	0.75371739	0.1933658	-0.2305269	-0.02686954
## 5	0.65731675	0.1933658	-0.2305269	-0.02686954
## 6	-0.09743372	0.1933658	-0.2305269	-0.02686954
## 7	1.30920349	0.1933658	-0.2305269	-0.02686954
##	mean_sev_ctl_wt	surv_sig	util_sig	qaly_sig
## 1	-0.01750482	FALSE	FALSE	FALSE
## 2	-0.01750482	TRUE	FALSE	TRUE
## 3	-0.01750482	FALSE	TRUE	TRUE
## 4	-0.01750482	TRUE	FALSE	TRUE
## 5	-0.01750482	TRUE	TRUE	TRUE
## 6	-0.01750482	TRUE	TRUE	FALSE
## 7	-0.01750482	TRUE	TRUE	TRUE
##		scenario_name	true_survival_effect	
## 1		Null	None	
## 2		Survival benefit only	Benefit	
## 3		Utility benefit only	None	
## 4		Early harm, late benefit	Small/moderate benefit	
## 5		Longer survival, worse QoL	Benefit	
## 6		Better QoL, shorter survival	Harm	
## 7		Modest survival + modest utility benefit	Small benefit	
##		true_utility_effect	expected_survival_test	
## 1		None	Not significant	
## 2		None	Likely significant	
## 3		Benefit	Not significant	
## 4		Early harm then late benefit	May be weak	
## 5		Harm	May be significant	
## 6		Benefit	May be significant in harmful direction	
## 7		Small benefit	May be weak	
##		expected_utility_test		
## 1		Not significant		
## 2		Not significant		
## 3		Likely significant		
## 4		May be weak because effects vary over time		
## 5		May detect worse utility		
## 6		May detect better utility		
## 7		May be weak		
##			expected_qaly_test	
## 1			Not significant	
## 2			Likely significant due to survival component	
## 3			Likely significant due to utility component	
## 4			Often stronger than individual tests, especially with time weighting	
## 5			May be attenuated because survival and utility oppose each other	
## 6			May be attenuated because survival and utility oppose each other	
## 7			Expected to be strongest because both domains improve modestly	
##			desired_message	
## 1			Type I error control	
## 2			QALY captures survival benefit while utility-only fails	
## 3			QALY captures utility benefit while survival-only fails	
## 4			QALY captures time-varying joint benefit	
## 5			Trade-off: composite endpoint reflects net benefit, not one marginal outcome	
## 6			Trade-off: composite endpoint reflects net benefit, not one marginal outcome	
## 7			Best example that QALY/HUS is more comprehensive	

8. Repeated simulation study / power table

```
run_power_study <- function(  
  scenarios = c("A", "B", "C", "D", "E", "F", "G"),  
  nsim = 30,  
  nperm = 100,  
  n0 = 150,  
  n1 = 150,  
  tt = 60,  
  alpha_level = 0.05,  
  seed = 100  
) {  
  
  all_results <- data.frame()  
  
  for (s in scenarios) {  
  
    cat("Running scenario:", s, "\n")  
  
    scenario_results <- data.frame(  
      scenario = s,  
      sim = 1:nsim,  
      surv_p = NA,  
      util_p = NA,  
      qaly_p = NA,  
      qaly_diff = NA  
    )  
  
    for (r in 1:nsim) {  
  
      one <- run_one_simulation(  
        scenario = s,  
        n0 = n0,  
        n1 = n1,  
        tt = tt,  
        nperm = nperm,  
        seed = seed + 1000 * match(s, scenarios) + r  
      )  
  
      scenario_results$surv_p[r] <- one$surv_p  
      scenario_results$util_p[r] <- one$util_p  
      scenario_results$qaly_p[r] <- one$qaly_p  
      scenario_results$qaly_diff[r] <- one$qaly_diff  
    }  
  
    all_results <- rbind(all_results, scenario_results)  
  }  
  
  summary_results <- aggregate(  
    cbind(  
      surv_sig = surv_p < alpha_level,  
      util_sig = util_p < alpha_level,  
      qaly_sig = qaly_p < alpha_level,
```



```

    qaly_diff = qaly_diff
  ) ~ scenario,
  data = all_results,
  FUN = mean
)

names(summary_results) <- c(
  "scenario",
  "survival_power",
  "utility_power",
  "qaly_power",
  "mean_qaly_diff"
)

return(list(
  raw = all_results,
  summary = summary_results
))
}
study <- run_power_study(nsim = 30, nperm = 100, seed= 100)

```

```

## Running scenario: A
## Running scenario: B
## Running scenario: C
## Running scenario: D
## Running scenario: E
## Running scenario: F
## Running scenario: G

```

```
study$summary
```

```

##  scenario survival_power utility_power qaly_power mean_qaly_diff
## 1      A      0.03333333  0.00000000 0.03333333    0.6346925
## 2      B      1.00000000  0.06666667 1.00000000    9.3362144
## 3      C      0.00000000  1.00000000 0.16666667    2.9714752
## 4      D      0.70000000  0.00000000 0.66666667    5.4906644
## 5      E      1.00000000  1.00000000 0.60000000    4.5945654
## 6      F      1.00000000  1.00000000 0.53333333   -4.4592526
## 7      G      1.00000000  1.00000000 1.00000000   12.1198675

```

Outputs generation

```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.1.3
```

```
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.1.3
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
library(tidyr)
```

```
## Warning: package 'tidyr' was built under R version 4.1.3
```

```
power_heatmap_data <- study$summary %>%
  select(
    scenario,
    survival_power,
    utility_power,
    qaly_power
  ) %>%
  pivot_longer(
    cols = c(survival_power, utility_power, qaly_power),
    names_to = "test",
    values_to = "rejection_rate"
  ) %>%
  mutate(
    test = recode(
      test,
      survival_power = "Survival-only",
      utility_power = "Utility-only",
      qaly_power = "QALY/HUS"
    ),
    scenario = factor(scenario, levels = c("A", "B", "C", "D", "E", "F", "G")),
    test = factor(test, levels = c("Survival-only", "Utility-only", "QALY/HUS")),
    label = sprintf("%.2f", rejection_rate)
  ) %>%
  mutate(
    scenario_label = recode(
      as.character(scenario),
      A = "A: Null",
      B = "B: Survival only",
      C = "C: Utility only",
      D = "D: Early harm/late benefit",
      E = "E: Longer survival/worse QoL",
      F = "F: Better QoL/shorter survival",
      G = "G: Both benefit"
    ),
    scenario_label = factor(
      scenario_label,
      levels = c(
```

```

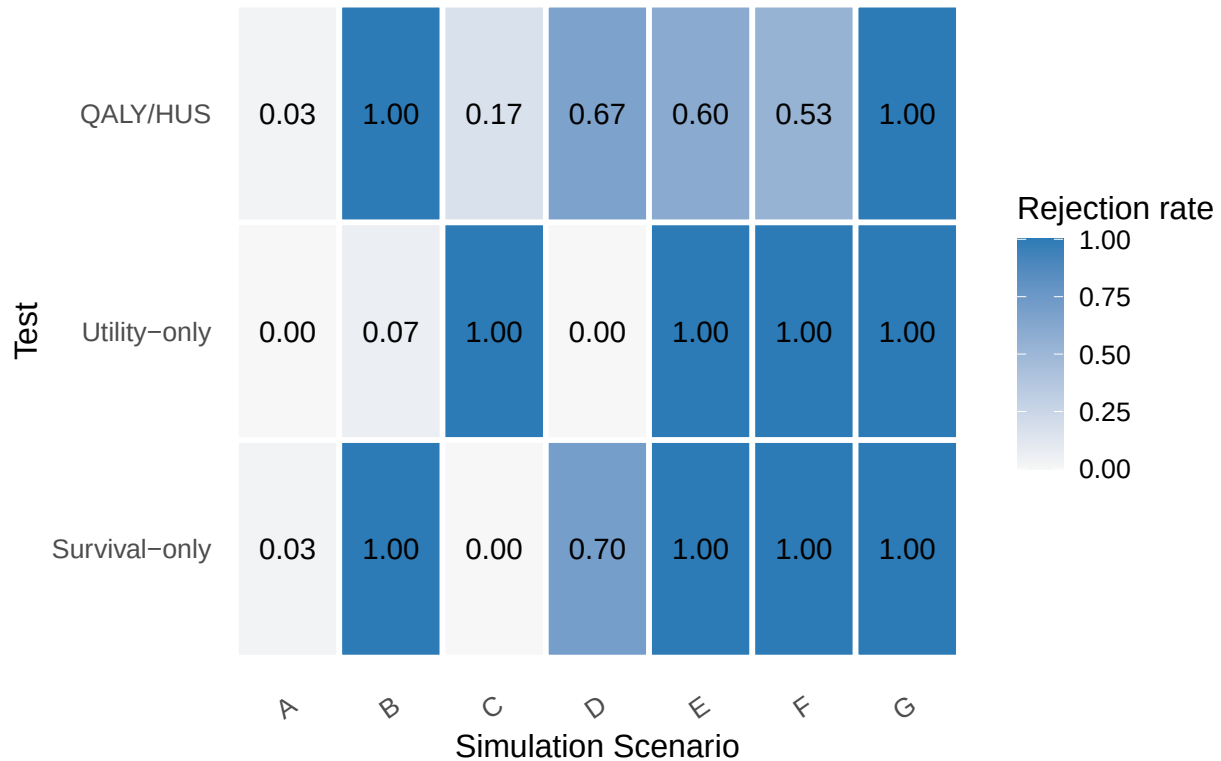
    "A: Null",
    "B: Survival only",
    "C: Utility only",
    "D: Early harm/late benefit",
    "E: Longer survival/worse QoL",
    "F: Better QoL/shorter survival",
    "G: Both benefit"
  )
)
)

fig_power_heatmap_labeled <- ggplot(
  power_heatmap_data,
  aes(x = scenario, y = test, fill = rejection_rate)
) +
  geom_tile(color = "white", linewidth = 1) +
  geom_text(aes(label = label), size = 3.8) +
  scale_fill_gradient(
    low = "#F7F7F7",
    high = "#2C7BB6",
    limits = c(0, 1),
    name = "Rejection rate"
  ) +
  labs(
    title = "Empirical Rejection Rates Across Simulation Scenarios",
    #subtitle = "Scenario A = type I error; Scenarios B-G = empirical power",
    x = "Simulation Scenario",
    y = "Test"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title = element_text(face = "bold"),
    axis.text.x = element_text(angle = 35, hjust = 1),
    panel.grid = element_blank()
  )

fig_power_heatmap_labeled

```

Empirical Rejection Rates Across Simulation Scenarios



```
heatmap_data <- one_run_results %>%
  select(scenario, surv_p, util_p, qaly_p) %>%
  pivot_longer(
    cols = c(surv_p, util_p, qaly_p),
    names_to = "test",
    values_to = "p_value"
  ) %>%
  mutate(
    test = recode(
      test,
      surv_p = "Survival-only",
      util_p = "Utility-only",
      qaly_p = "QALY/HUS"
    ),
    scenario = factor(scenario, levels = c("A", "B", "C", "D", "E", "F", "G")),
    test = factor(test, levels = c("Survival-only", "Utility-only", "QALY/HUS")),
    result = ifelse(p_value < 0.05, "Significant", "Not significant"),
    p_label = ifelse(p_value < 0.001, "<0.001", sprintf("%.3f", p_value))
  )

fig_heatmap <- ggplot(
  heatmap_data,
  aes(x = scenario, y = test, fill = result)
) +
  geom_tile(color = "white", linewidth = 1) +
  geom_text(aes(label = p_label), size = 4) +
```

```

scale_fill_manual(
  values = c(
    "Significant" = "#2C7BB6",
    "Not significant" = "#D9D9D9"
  )
) +
labs(
  title = "Significance Pattern Across Simulation Scenarios",
  x = "Simulation Scenario",
  y = "Test",
  fill = "Result"
) +
theme_minimal(base_size = 13) +
theme(
  legend.position = "bottom",
  plot.title = element_text(face = "bold"),
  panel.grid = element_blank()
)
fig_heatmap

```

